

高等計算機演算法

term project

學號：M11202169

姓名：鍾坤佑

Design RSA	3
1. 128-bit SIPO (Serial-In, Parallel-Out) Shift Register	3
2. 128-bit PISO (Parallel-In, Serial-Out) Shift Register	4
3. Modular Multiplication Module	4
4. Modular Exponential Module	6
5. RSA Encrypt 128-bit	7
6. RSA Decrypt 128-bit	8
7. Datapath	8
8. Controller	9
9. Complete RSA System	10
Verilog Code and Simulation	11
1. 128-bit SIPO (Serial-In, Parallel-Out) Shift Register	11
2. 128-bit PISO (Parallel-In, Serial-Out) Shift Register	13
3. Modular Multiplication Module	15
4. Modular Exponential Module	18
5. RSA Encrypt 128-bit	23
6. RSA Decrypt 128-bit	26
7. Datapath	30
8. Controller	33
9. Complete RSA System	38
Hardware cost	41
Discussion	42

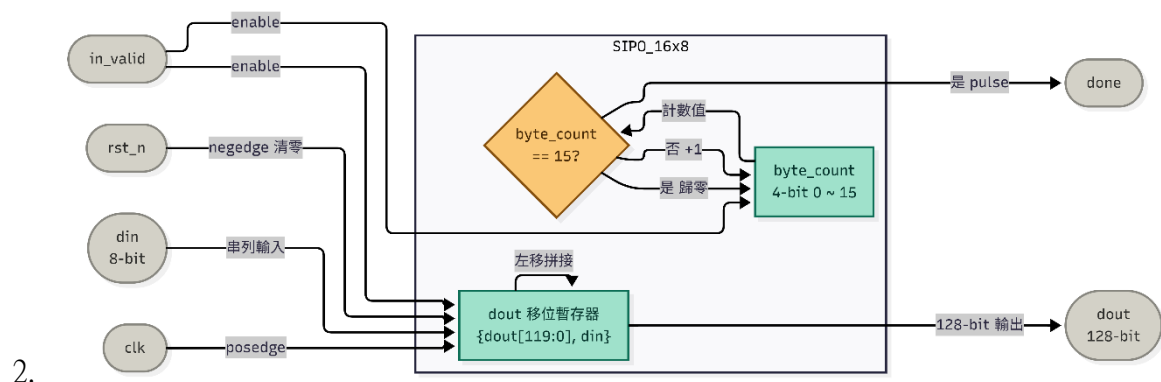
Design RSA

這次的 project 我實作了一個 128-bit 的 RSA 加解密硬體 IP。在整體的系統設計上，因為金鑰生成的數學計算比較繁雜，所以我先在軟體端使用 python 來處理前置作業：包含找尋兩個大質數 p 和 q 、計算出模數 n ，並選定標準的 $e = 65537$ 作為公鑰，最後計算並推導出私鑰 d 。將這些參數預先準備好後，再交給硬體來執行真正龐大的加密與解密運算。

在硬體架構的運作上，資料流分為三個階段。首先透過輸入介面，將 128-bit 的資料載入到內部暫存器。接著會進入核心的模指數運算模組，為了在硬體資源上取得理想的平衡，我沒有使用複雜的演算法或大型的硬體乘法器，而是設計了一個簡單且直觀的 two adder 架構。這代表核心的運算完全仰賴最基礎的加減法與位移操作來進行計算。這樣的做法雖然在週期數上會比較長，但能非常有效地節省硬體的面積開銷。

最後，當核心運算完成後，就會將 128-bit 的結果輸出。在時序控制方面，整個運算過程由內部的 controller (FSM) 來嚴格控制 two adder 的計算進度。完成一次完整的 RSA 加解密流程（包含資料輸入、核心運算與資料輸出）。

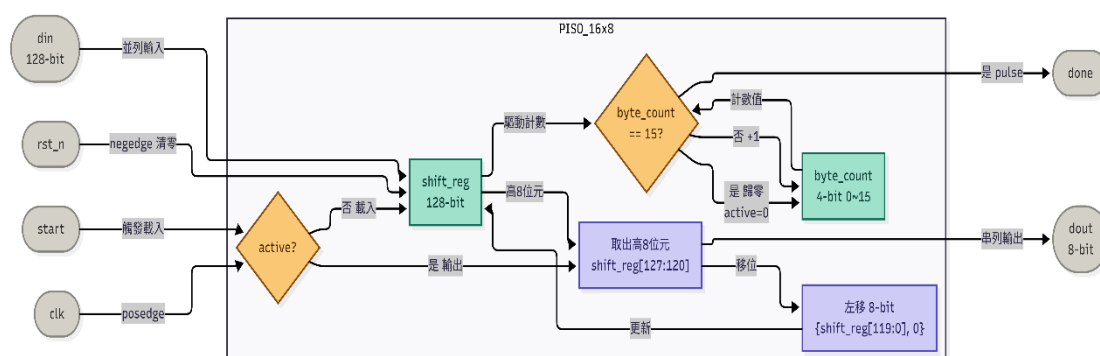
1. 128-bit SIPO (Serial-In, Parallel-Out) Shift Register



首先這是一個負責將 serial-in input 轉換為 parallel-out output 的資料接收模組。在運作機制上，它每次會接收 8-bit 的資料，並且，模組內部配置了一個 4-bit 的計數器，只要確認當下的輸入訊號有效（in_valid=1），電路就會把這 8-bit 的新資料推進 128-bit 暫存器的最低位元處，同時將原本的舊資料向左移位。

每成功接收一次 8-bit，計數器就會進行累加，當計數器數到 15，代表已經連續收滿 16 筆資料，一筆完整的 128-bit 資料就順利拼接完成了，這個時候，模組會拉高 done 訊號，通知後續的運算模組資料已經準備就緒，可以開始進行加密的運算了。

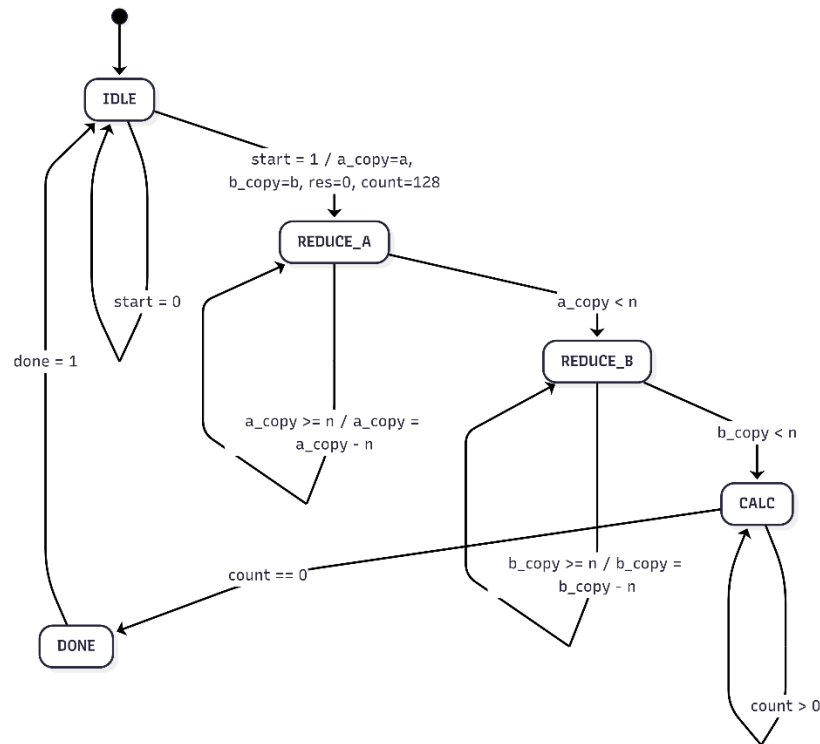
2. 128-bit PISO (Parallel-In, Serial-Out) Shift Register



接著，這是一個負責將並列輸入轉換為串列輸出的資料傳輸模組。當內部的 two adder 核心完成所有的 RSA 加解密疊代運算後，會給出一個 start 啟動訊號，這時，模組內部的 active 狀態就會被觸發，並立刻將這 128-bit 的完整運算結果，一次性的載入到內部的 shift register 當中。

在接下來的傳輸階段，模組每次會精準截取暫存器最高位的 8 個 bit [127:120] 作為當次的輸出資料 dout，同時將暫存器內剩下的資料統一向左移位 8 個 bit。我們同樣搭配了一個 4-bit 的計數器來掌控傳輸進度，當計數器數滿 15，也就是連續輸出 16 次、把 128-bit 的資料全部安全送達後，系統就會自動解除 active 狀態，並拉高 done 訊號，代表這筆 RSA 資料的處理與輸出流程已經完成。

3. Modular Multiplication Module



接著，這個模組負責執行 RSA 運算中最核心的模乘法 (Modular Multiplication)。為了省下大型模乘硬體的面積，我選擇設計一個 two adder 的運算架構，裡面包含五個狀態的 FSM。

整個運算流程分為三個主要階段：

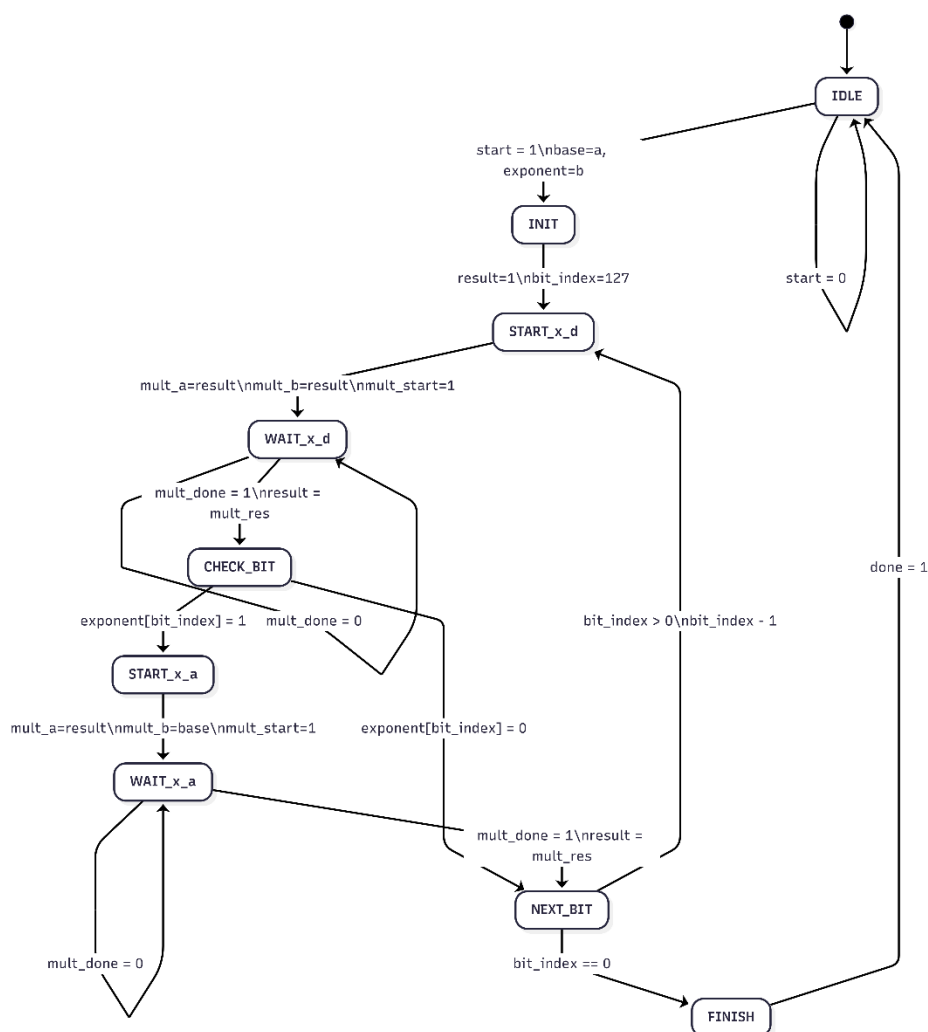
- 前置正規化 (REDUCE_A & REDUCE_B)：在運算剛啟動時，電路會先確保輸入的被乘數 a 與乘數 b 都嚴格小於模數 n ，如果發現數值大於或等於 n ，就會先減去 n ，防止後續在做疊代加法時發生溢位。
- 核心疊代 (CALC)：這是整個模組的關鍵所在，固定會執行 128 個 clock cycle，在這裡我配置了兩個平行的加法器與位移電路：

第一個 adder(結果累積)：負責檢查乘數 b 的最低位元 ($b_copy[0]$)。如果為 1，就把當下的被乘數 a 加進結果 res 裡，若相加後的結果大於或等於模數 n ，就立刻減去 n 以維持在模數範圍內。

第二個 adder(被乘數翻倍)：同步將被乘數 a 向左位移 1 bit，等同於乘以 2，同樣地，只要這個翻倍的數值大於或等於 n ，就立刻減去 n 。與此同時，乘數 b 會向右位移 1 bit，為下一個 cycle 的最低位元檢查做準備。

- 完成輸出 (DONE)：經過 128 輪的運算後 ($count==0$)，最終 128-bit 的模乘結果就完成了，這時狀態機會切換並拉高 $done$ 訊號。

4. Modular Exponential Module



接著是負責整合核心運算的 `mod_exp_128` 模組。由於 RSA 的金鑰指數 (exponent) 高達 128-bit，如果用傳統的方法慢慢乘，會消耗天文數字般的運算週期。因此，我在這裡實作了經典的 Square-and-Multiply 演算法，也就是由左至右的二進制指數掃描法。

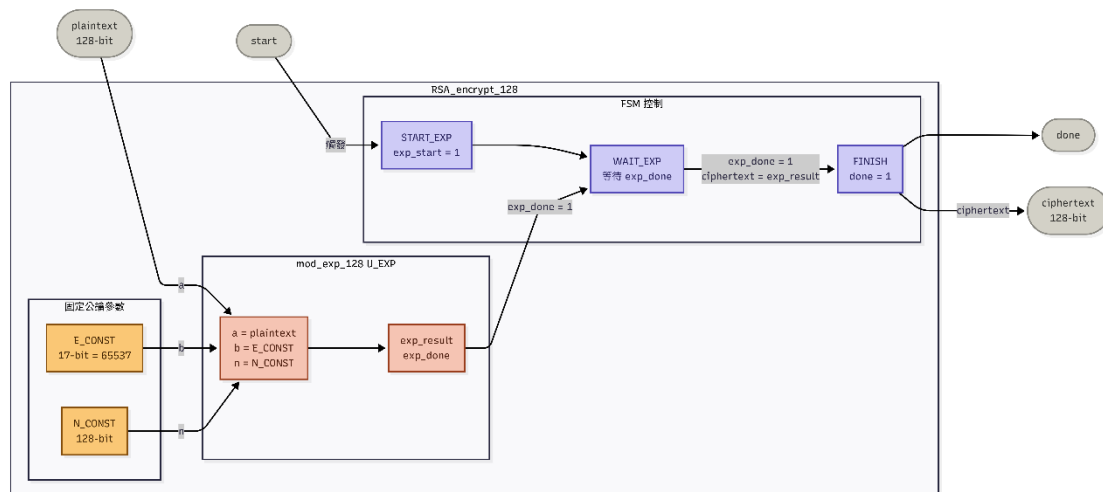
在硬體架構上，我設計了一個包含 9 個狀態的 FSM，用來精準調度剛才設計的模乘模組。整體的運作流程如下：

- 初始化與掃描 (INIT)：運算開始時，我將結果暫存器預設為 1，並設定一個指標 (`bit_index = 127`)，準備從指數 MSB 一路往下掃到 LSB。
- 平方階段 (START_x_d)：無論當下的指數 bit 是多少，每個循環都會先呼叫

前一個的模乘模組，將當前的結果進行平方，也就是把輸入參數都設定為目前的 result 來進行運算。

- 條件乘法階段 (CHECK_BIT & START_x_a)：等待平方運算完成後，狀態機會檢查當下掃描到的指數 bit 是否為 1。如果是 1，就再次啟動模乘模組，把當前的結果乘上最原始的底數 base。如果該 bit 是 0，則直接跳過這個步驟。
- 遞迴與輸出 (NEXT_BIT & FINISH)：完成上述動作後，指標會減 1 並進入下一個位元的運算。當 128 個 bit 全部處理完畢，最終的模冪運算結果就出爐了，此時便拉高 done 訊號結束流程。

5. RSA Encrypt 128-bit

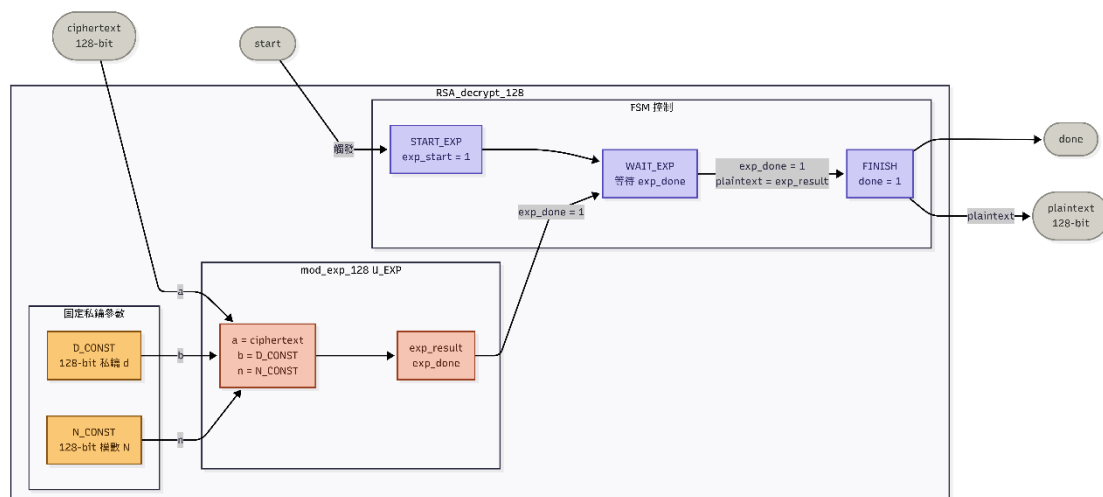


接著是 RSA 加密模組，它的主要任務是接收 128-bit 的明文 (plaintext)，並將其轉換成密文 (ciphertext) 輸出。在架構設計上，我預先把 python 內算好的模數 n 以及標準的公開金鑰 $e = 65537$ ，直接被宣告為硬體內部的固定參數 (N_CONST 與 E_CONST)。這樣就不需要再耗費任何額外資源去處理金鑰的生成，當然也可以另外實作 key generation 的硬體模組，只不過這需要 LFSR 來產生大的隨機奇數，再加上 Miller-Rabin primality test 來判斷這個大奇數是否為質數，通過多輪檢測之後就幾乎可以確認這個大數為質數了。

在此加密模組的控制邏輯方面，我使用了一個 4 狀態的 FSM (IDLE, START_EXP, WAIT_EXP, FINISH) 來進行管理。當接收到啟動訊號後，狀態機會把明文、 e 與 n 作為參數，傳遞給 mod_exp_128 模指數運算核心。接著，電路會進入等待狀態。一旦疊代運算完成並收到 exp_done 訊號，這個模組就會將結果

存到 ciphertext 輸出端，並發出 done 訊號，完成加密流程。

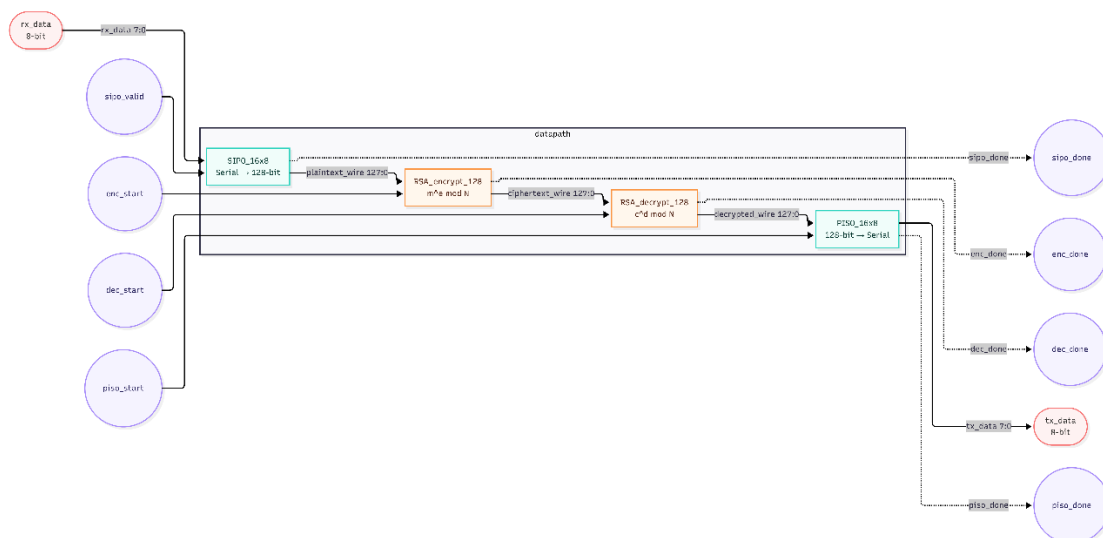
6. RSA Decrypt 128-bit



接著是 RSA 解密模組。同樣的，這裡的模數 n ，以及我算出的私鑰 d ，同樣直接宣告為硬體內部的固定參數。

在運作邏輯上，它和加密模組有相同的 4 狀態 FSM。當收到啟動訊號後，狀態機會把密文、私鑰 d 與模數 n 餵給 mod_exp_128 模指數運算模組。等到完成疊代運算並回傳完成訊號 (exp_done) 後，就會把還原好的明文存到輸出端，並拉高 done 訊號，完成解密流程。

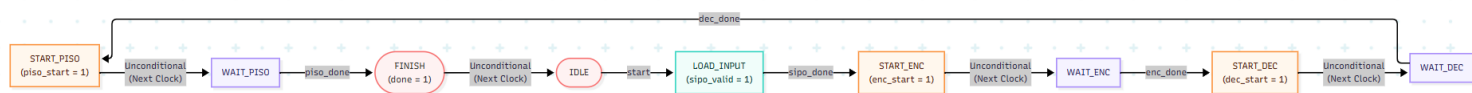
7. Datapath



接著是整個專案的 datapath。在這個模組中，我將前面獨立設計模組全部整合起來。首先，外部資料透過 8-bit 的 rx_data 進入前端的 SIPO 模組，被組裝成 128-bit 的明文 (plaintext_wire) 後，接著，會送入 RSA_encrypt_128 進行加密，產生出 128-bit 的密文 (ciphertext_wire)，且為了直接在硬體上驗證演算法的正確性，我將加密模組的輸出，對接到 RSA_decrypt_128 直接進行解密，將其還原成 128-bit 的明文 (decrypted_wire)，最後，這筆資料會傳到後端的 PISO 模組，重新切分成 8-bit 的 tx_data 依序傳送出去。

此外，再透過外圍的 controller 來控制各個模組之間的 handshake 訊號，就可以精準控制每階段的時序。

8. Controller

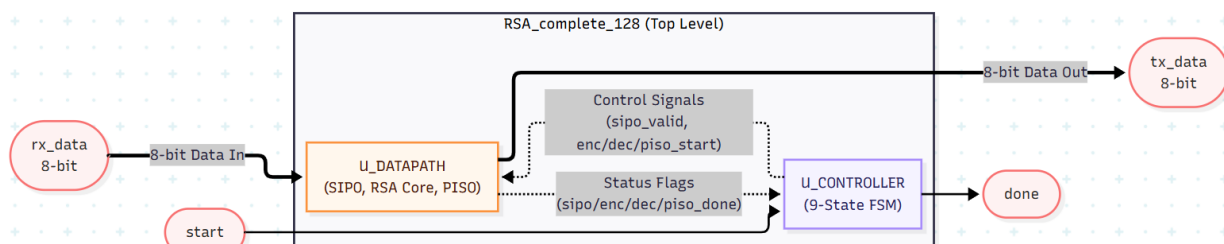


接著是 controller 的部分。它的任務是精準調度 datapath 裡面各個模組的控制訊號，確保資料能順暢地完成整個流程。在設計上，我用一個 7 個狀態的 FSM 來進行控制。當外部給予 start 啟動訊號後，就會開始嚴格依照順序執行任務：

- 載入階段 (LOAD_INPUT)：觸發前端 SIPO 模組開始接收 16 bytes 的外部資料，並等待其回傳 sipo_done 完成訊號。
- 加密階段 (START_ENC / WAIT_ENC)：SIPO 收滿資料後，立刻拉高 enc_start 啟動加密模組，接著進入等待狀態，直到加密完成 (enc_done)。
- 解密驗證 (START_DEC / WAIT_DEC)：這是為了驗證正確性所設計的連續動作。加密完成後，系統會啟動解密引擎 (dec_start)，將剛剛算出的密文直接還原，同樣等待解密完成 (dec_done) 訊號，最終確認解出來的明文是否與輸入的一致。
- 輸出階段 (OUTPUT_DATA)：解密成功後，指示後續的 PISO 模組將 16 bytes 的明文依序往外送。當資料全數送達 (piso_done)，主控模組便會拉高 top

module 的 done 訊號，並回到 IDLE 狀態，準備下一筆加解密操作。

9. Complete RSA System



這是我為此專案設計的 top 模組 RSA_complete_128。在這裡，我將負責處理控制訊號的 controller 與負責運算的 datapath 進行了最終的實例化與對接。

只要提供最基礎的 clk 與 rst_n，拉高 start 啟動訊號，接著把資料按 8-bit 的寬度依序從 rx_data 餵進來，等系統內部自動跑完複雜的接收、加密、解密驗證流程後，最後只要看到 done 訊號亮起，就可以直接從 tx_data 把 8-bit 的結果讀出來。

Verilog Code and Simulation

1. 128-bit SIPO (Serial-In, Parallel-Out) Shift Register

Verilog :

```
23 module SIPO_16x8(  
24  
25     input wire clk,  
26     input wire rst_n,  
27  
28     input wire in_valid,  
29     input wire [7:0] din,  
30  
31     output reg [127:0] dout,  
32     output reg done  
33 );  
34  
35     reg [3:0] byte_count;  
36  
37     always @(posedge clk or negedge rst_n) begin  
38  
39         if(!rst_n) begin  
40  
41             dout <= 128'd0;  
42             byte_count <= 4'd0;  
43             done <= 1'b0;  
44         end  
45         else begin  
46  
47             done <= 1'b0;  
48  
49             if(in_valid) begin  
50  
51                 dout <= {dout[119:0], din};  
52  
53                 if(byte_count == 4'd15) begin  
54  
55                     byte_count <= 4'd0;  
56                     done <= 1'b1;  
57                 end  
58                 else begin  
59  
60                     byte_count <= byte_count + 1'b1;  
61                 end  
62             end  
63         end  
64     end  
65 endmodule
```

Testbench :

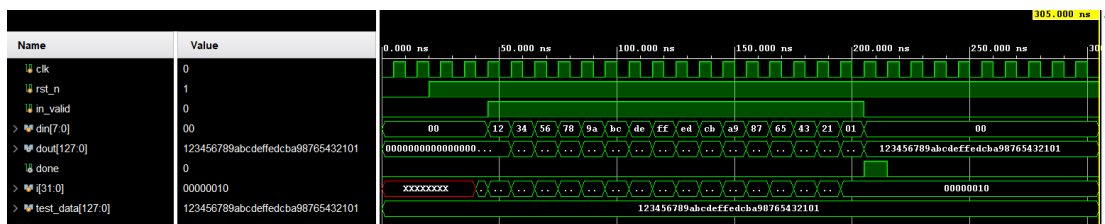
```

23 module tb_SIPO_16x8;
24     reg clk;
25     reg rst_n;
26     reg load;
27     reg [7:0] din;
28     wire [127:0] dout;
29     wire done;
30
31     SIPO_16x8 uut (
32         .clk(clk),
33         .rst_n(rst_n),
34         .load(load),
35         .din(din),
36         .dout(dout),
37         .done(done)
38     );
39
40     always #10 clk = ~clk;
41     initial begin
42         clk = 0;
43         rst_n = 0;
44         load = 0;
45         din = 8'h00;
46
47         #100;
48         rst_n = 1;
49
50         // 依序送 16 個 byte
51         repeat (16) begin
52             @(negedge clk);
53             load = 1;
54             din = $random; // 隨機送一個 byte
55         end
56
57         @(negedge clk);
58         load = 0;
59
60         #100;
61         $finish;
62     end
63 endmodule

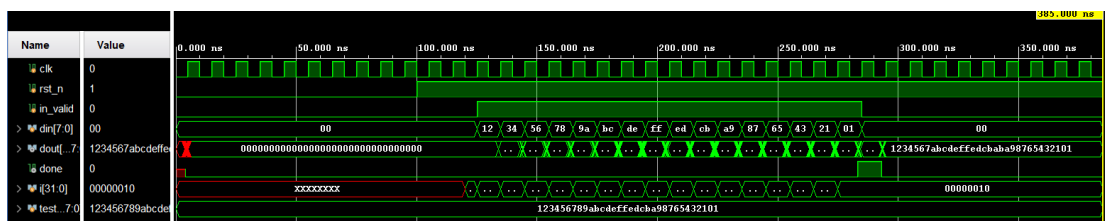
```

Simulation

Behavior Simulation :



Post-Route Simulation :



2. 128-bit PISO (Parallel-In, Serial-Out) Shift Register

Verilog :

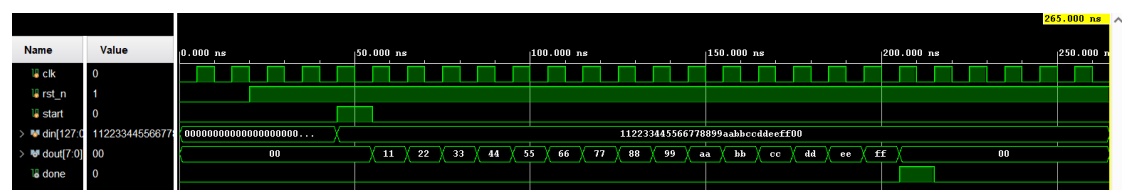
```
23 module PISO_16x8(  
24  
25     input wire clk,  
26     input wire rst_n,  
27  
28     input wire start,  
29  
30     input wire [127:0] din,  
31  
32     output reg [7:0] dout,  
33     output reg done  
34 );  
35  
36     reg [127:0] shift_reg;  
37     reg [3:0] byte_count;  
38     reg active;  
39  
40     always @(posedge clk or negedge rst_n) begin  
41  
42         if(!rst_n) begin  
43  
44             shift_reg <= 128'd0;  
45             byte_count <= 4'd0;  
46             dout <= 8'd0;  
47             done <= 1'b0;  
48             active <= 1'b0;  
49         end  
50     else begin  
51  
52         done <= 1'b0;  
53  
54         //=====  
55         // Start Serialization  
56         //=====  
57  
58         if(start && !active) begin  
59  
60             shift_reg <= din;  
61             byte_count <= 4'd0;  
62             active <= 1'b1;  
63         end  
64  
65         //=====  
66         // Output Bytes  
67         //=====  
68  
69         else if(active) begin  
70  
71             dout <= shift_reg[127:120];  
72             shift_reg <= {shift_reg[119:0], 8'd0};  
73  
74             if(byte_count == 4'd15) begin  
75  
76                 byte_count <= 4'd0;  
77                 active <= 1'b0;  
78                 done <= 1'b1;  
79             end  
80             else begin  
81  
82                 byte_count <= byte_count + 1'b1;  
83             end  
84         end  
85     end  
86 end  
87 endmodule
```

Testbench :

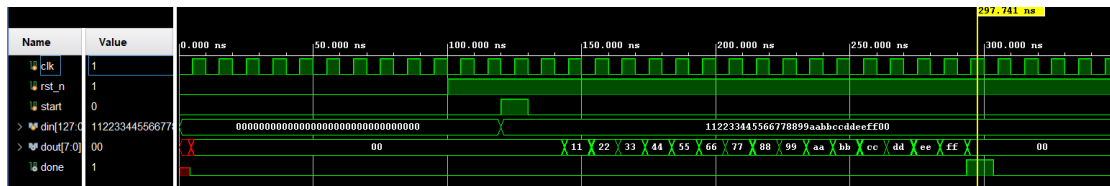
```
23 module tb_PISO_16x8();
24
25     // 宣告連接 DUT 的訊號
26     reg clk;
27     reg rst_n;
28     reg start;
29     reg [127:0] din;
30
31     wire [7:0] dout;
32     wire done;
33
34     PISO_16x8 uut (
35         .clk(clk),
36         .rst_n(rst_n),
37         .start(start),
38         .din(din),
39         .dout(dout),
40         .done(done)
41     );
42
43     initial begin
44         clk = 0;
45         forever #5 clk = ~clk;
46     end
47
48     initial begin
49
50         rst_n = 0;
51         start = 0;
52         din = 128'd0;
53
54         #100;
55         rst_n = 1;
56         #20;
57
58         @(negedge clk);
59         din = 128'h11_22_33_44_55_66_77_88_99_AA_BB_CC_DD_EE_FF_00;
60         start = 1;
61
62         @(negedge clk);
63         start = 0;
64
65         wait(done);
66         @(negedge clk);
67
68         #50;
69         $finish;
70     end
71 endmodule
```

Simulation :

Behavior Simulation :



Post-Route Simulation :



3. Modular Multiplication Module

Verilog :

```

23 module mod_mult_128(
24     input wire clk,
25     input wire rst_n,
26     input wire start,
27     input wire [127:0] a,    // 被乘數
28     input wire [127:0] b,    // 乘數
29     input wire [127:0] n,    // 模數
30     output reg [127:0] res,   // 結果
31     output reg done
32 );
33
34 // 狀態定義
35 parameter IDLE = 3'd0;
36 parameter REDUCE_A = 3'd1;
37 parameter REDUCE_B = 3'd2;
38 parameter CALC = 3'd3;
39 parameter DONE = 3'd4;
40
41 reg [2:0] state;
42 reg [127:0] a_copy;
43 reg [127:0] b_copy;
44 reg [7:0] count;
45
46 // 中間運算暫存，多出一位（129 bits）用來判斷加法後的溢位與比較
47 reg [128:0] next_res;
48 reg [128:0] next_a;
49
50 always @(posedge clk or negedge rst_n) begin
51     if (!rst_n) begin
52         state <= IDLE;
53         res <= 128'b0;
54         done <= 1'b0;
55         count <= 8'b0;
56         a_copy <= 128'b0;
57         b_copy <= 128'b0;
58     end else begin
59         case (state)
60             IDLE: begin
61                 done <= 1'b0;
62                 if (start) begin
63                     a_copy <= a;
64                     b_copy <= b;
65                     res <= 128'b0;
66                     count <= 8'd128; // 數 128 輪
67                     state <= REDUCE_A;

```

```

68     end
69 end
70
71 REDUCE_A: begin // 確保 a < n
72     if(a_copy >= n)
73         a_copy <= a_copy - n;
74     else
75         state <= REDUCE_B;
76     end
77
78 REDUCE_B: begin // 確保 b < n
79     if(b_copy >= n)
80         b_copy <= b_copy - n;
81     else
82         state <= CALC;
83     end
84
85 CALC: begin
86     if (count == 8'd0) begin
87         state <= DONE;
88     end else begin
89         // --- 第一個 Adder: 處理累積和 (Result) ---
90         if (b_copy[0]) begin
91             next_res = res + a_copy;
92             // 若結果 >= n, 則減去 n (維持在模數範圍內)
93             if (next_res >= n)
94                 res <= next_res - n;
95             else
96                 res <= next_res[127:0];
97         end
98
99         // --- 第二個 Adder: 處理 a 的幕次方 mod n
100        next_a = a_copy << 1;
101        if (next_a >= n)
102            a_copy <= next_a - n;
103        else
104            a_copy <= next_a[127:0];
105
106        // 位移 b, 準備下一輪
107        b_copy <= b_copy >> 1;
108        count <= count - 8'd1;
109    end
110 end
111
112 DONE: begin
113     done <= 1'b1;
114     state <= IDLE;
115 end
116
117 default: state <= IDLE;
118 endcase
119 end
120 end
121 endmodule

```

Testbench :


```

23 module tb_mod_mult_128;
24
25     // Inputs
26     reg clk;
27     reg rst_n;
28     reg start;
29     reg [127:0] a;
30     reg [127:0] b;
31     reg [127:0] n;
32
33     // Outputs
34     wire [127:0] res;
35     wire done;
36
37     mod_mult_128 uut (
38         .clk(clk),
39         .rst_n(rst_n),
40         .start(start),
41         .a(a),
42         .b(b),
43         .n(n),
44         .res(res),
45         .done(done)
46     );
47
48     initial begin
49         clk = 0;
50         forever #5 clk = ~clk;
51     end
52
53     initial begin
54         rst_n = 0;
55         start = 0;
56         a = 0;
57         b = 0;
58         n = 0;
59
60         #20;
61         rst_n = 1;
62         #20;
63
64         // 第一組測試
65         // 10 * 5 mod 7 = 1
66         a = 128'd10;
67         b = 128'd5;
68         n = 128'd7;
69         start = 1;
70         #10;
71         start = 0;
72
73         // 等待運算完成
74         wait(done);
75         #50;
76
77         // 第二組測試
78         // 11 * 3 mod 7 = 5
79         a = 128'd11;
80         b = 128'd3;
81         n = 128'd7;
82         start = 1;
83         #10;
84         start = 0;
85
86         wait(done);
87         #50;

```

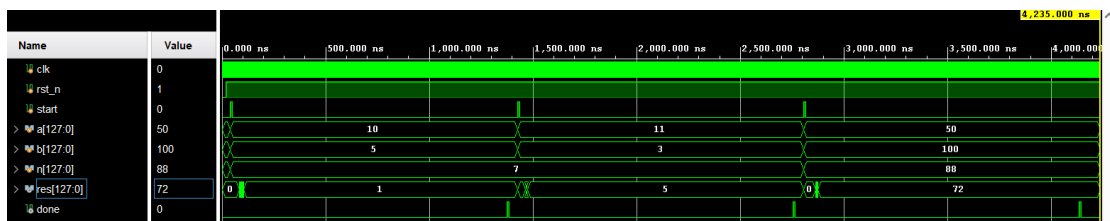
```

88
89 // 第三組測試
90 // 50 * 100 mod 88 = 72
91 a = 128'd50;
92 b = 128'd100;
93 n = 128'd88;
94 start = 1;
95 #10;
96 start = 0;
97
98 wait(done);
99 #100;
100
101 $finish;
102 end
103 endmodule

```

Simulation :

Behavior Simulation :



4. Modular Exponential Module

Verilog :

```

23 module mod_exp_128( // a^b mod n
24     input wire clk,
25     input wire rst_n,
26     input wire start,
27
28     input wire [127:0] a,
29     input wire [127:0] b,
30     input wire [127:0] n,
31
32     output reg [127:0] result,
33     output reg done
34 );
35
36 //=====
37 // FSM state
38 //=====
39
40 parameter IDLE      = 4'd0;
41 parameter INIT      = 4'd1;
42 parameter START_x_d = 4'd2;
43 parameter WAIT_x_d  = 4'd3;
44 parameter CHECK_BIT = 4'd4;
45 parameter START_x_a = 4'd5;

```

```

46 parameter WAIT_x_a      = 4'd6;
47 parameter NEXT_BIT      = 4'd7;
48 parameter FINISH        = 4'd8;
49
50 reg [3:0] state;
51
52 //=====
53 // Registers
54 //=====
55
56 reg [127:0] base;
57 reg [127:0] exponent;
58
59 reg [7:0] bit_index;
60
61 reg mult_start;
62
63 reg [127:0] mult_a;
64 reg [127:0] mult_b;
65
66 wire [127:0] mult_res;
67 wire mult_done;
68
69 //=====
70 // Modular Multiplier Instance
71 //=====
72
73 mod_mult_128 U_MULT (
74     .clk(clk),
75     .rst_n(rst_n),
76     .start(mult_start),
77     .a(mult_a),
78     .b(mult_b),
79     .n(n),
80     .res(mult_res),
81     .done(mult_done)
82 );
83
84 //=====
85 // FSM
86 //=====
87
88 always @(posedge clk or negedge rst_n) begin
89     if(!rst_n) begin
90         state <= IDLE;
91         result <= 0;
92         done <= 0;
93         base <= 0;
94         exponent <= 0;
95         bit_index <= 0;
96         mult_start <= 0;
97         mult_a <= 0;
98         mult_b <= 0;
99     end
100     else begin
101         case(state)
102
103             //=====
104             // IDLE
105             //=====
106             IDLE: begin
107
108                 done <= 0;
109                 mult_start <= 0;
110

```

```

111         if(start) begin
112             base <= a;
113             exponent <= b;
114
115             state <= INIT;
116         end
117     end
118
119     //=====
120     // INIT
121     //=====
122     INIT: begin
123
124         result <= 128'd1;
125         bit_index <= 8'd127;
126
127         state <= START_x_d;
128     end
129
130     //=====
131     // START_x_d
132     // result = result * result mod n
133     //=====
134     START_x_d: begin
135
136         mult_a <= result;
137         mult_b <= result;
138         mult_start <= 1'b1;
139
140         state <= WAIT_x_d;
141     end
142
143     //=====
144     // WAIT_x_d
145     //=====
146     WAIT_x_d: begin
147
148         mult_start <= 1'b0;
149
150         if(mult_done) begin
151             result <= mult_res;
152
153             state <= CHECK_BIT;
154         end
155     end
156
157     //=====
158     // CHECK_BIT
159     //=====
160     CHECK_BIT: begin
161
162         if(exponent[bit_index]) begin // exponent[127] ~ exponent[0]
163             state <= START_x_a;
164         end
165         else begin
166             state <= NEXT_BIT;
167         end
168     end

```

```

170 //=====
171 // START_x_a
172 // result = result * base mod n
173 //=====
174 START_x_a: begin
175
176     mult_a <= result;
177     mult_b <= base;
178     mult_start <= 1'b1;
179
180     state <= WAIT_x_a;
181 end

183 //=====
184 // WAIT_x_a
185 //=====
186 WAIT_x_a: begin
187
188     mult_start <= 1'b0;
189
190     if(mult_done) begin
191         result <= mult_res;
192
193         state <= NEXT_BIT;
194     end
195 end

197 //=====
198 // NEXT_BIT
199 //=====
200 NEXT_BIT: begin
201
202     if(bit_index == 0) begin
203         state <= FINISH;
204     end
205     else begin
206         bit_index <= bit_index - 1;
207
208         state <= START_x_d;
209     end
210
211 end

213 //=====
214 // FINISH
215 //=====
216 FINISH: begin
217
218     done <= 1'b1;
219
220     state <= IDLE;
221 end

223 default: begin
224     state <= IDLE;
225 end
226 endcase
227 end
228 end
229 endmodule

```

Testbench :

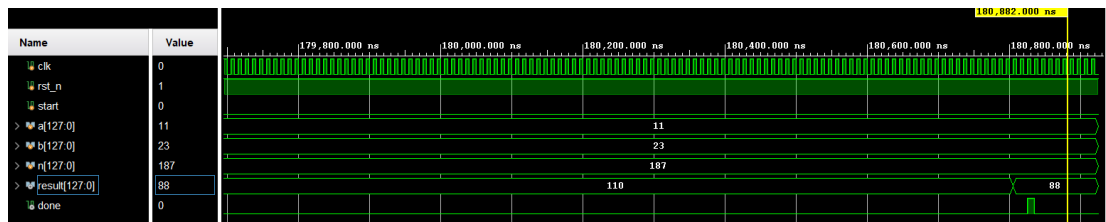
```

23 module tb_mod_exp_128;
24     reg clk;
25     reg rst_n;
26     reg start;
27
28     reg [127:0] a;
29     reg [127:0] b;
30     reg [127:0] n;
31
32     wire [127:0] result;
33     wire done;
34
35     mod_exp_128 DUT (
36         .clk(clk),
37         .rst_n(rst_n),
38         .start(start),
39
40         .a(a),
41         .b(b),
42         .n(n),
43
44         .result(result),
45         .done(done)
46     );
47
48     initial begin
49         clk = 0;
50         forever #5 clk = ~clk; // 10ns period
51     end
52
53     initial begin
54
55         rst_n = 0;
56         start = 0;
57
58         a = 0;
59         b = 0;
60         n = 0;
61
62         #20;
63         rst_n = 1;
64
65         //=====
66         // Example:
67         // 11^23 mod 187 = 88
68         //=====
69
70         #20;
71         a = 128'd11;
72         b = 128'd23;
73         n = 128'd187;
74
75         // start pulse
76         start = 1;
77         #10;
78         start = 0;
79
80         wait(done);
81
82         #100;
83         $finish;
84     end
85 endmodule

```

Simulation :

Behavior Simulation :



5. RSA Encrypt 128-bit

Verilog :

```

23 module RSA_encrypt_128(
24
25     input wire clk,
26     input wire rst_n,
27     input wire start,
28
29     input wire [127:0] plaintext,
30
31     output reg [127:0] ciphertext,
32     output reg done
33 );
34
35
36 //=====
37 // Fixed RSA Public Key
38 //=====
39
40 parameter [127:0] N_CONST =
41     128'h9195a8ae3168963d09492e2d9420b23;
42
43 parameter [16:0] E_CONST =
44     17'd65537;
45
46 //=====
47 // FSM
48 //=====
49
50 parameter IDLE      = 2'd0;
51 parameter START_EXP = 2'd1;
52 parameter WAIT_EXP  = 2'd2;
53 parameter FINISH    = 2'd3;
54 reg [1:0] state;
55
56 //=====
57 // mod_exp interface
58 //=====
59
60 reg exp_start;
61 wire [127:0] exp_result;
62 wire exp_done;

```

```

64 //=====
65 // Modular Exponentiation
66 //=====
67
68 mod_exp_128 U_EXP (
69     .clk(clk),
70     .rst_n(rst_n),
71     .start(exp_start),
72
73     .a(plaintext),
74     .b(E_CONST),
75     .n(N_CONST),
76
77     .result(exp_result),
78     .done(exp_done)
79 );
80
81 //=====
82 // FSM
83 //=====
84
85 always @(posedge clk or negedge rst_n) begin
86
87     if(!rst_n) begin
88
89         state <= IDLE;
90
91         ciphertext <= 0;
92         done <= 0;
93
94         exp_start <= 0;
95     end
96     else begin
97
98         case(state)
99
100             //=====
101             // IDLE
102             //=====
103
104             IDLE: begin
105
106                 done <= 0;
107                 exp_start <= 0;
108
109                 if(start) begin
110
111                     state <= START_EXP;
112                 end
113             end
114
115             //=====
116             // START modular exponentiation
117             //=====
118
119             START_EXP: begin
120
121                 exp_start <= 1'b1;
122
123                 state <= WAIT_EXP;
124             end
125

```



```

126 //=====
127 // WAIT
128 //=====
129
130 WAIT_EXP: begin
131
132     exp_start <= 1'b0;
133
134     if(exp_done) begin
135
136         ciphertext <= exp_result;
137
138         state <= FINISH;
139     end
140 end
141
142 //=====
143 // FINISH
144 //=====
145
146 FINISH: begin
147
148     done <= 1'b1;
149
150     state <= IDLE;
151 end
152
153 default: begin
154
155     state <= IDLE;
156 end
157 endcase
158 end
159 end
160 endmodule

```

Testbench :

```

23 module tb_RSA_encrypt_128;
24     reg clk;
25     reg rst_n;
26     reg start;
27
28     reg [127:0] plaintext;
29
30     wire [127:0] ciphertext;
31     wire done;
32
33     RSA_encrypt_128 DUT (
34
35         .clk(clk),
36         .rst_n(rst_n),
37         .start(start),
38
39         .plaintext(plaintext),
40
41         .ciphertext(ciphertext),
42         .done(done)
43     );
44
45     initial begin
46         clk = 0;
47         forever #5 clk = ~clk;
48     end
49

```



```

23 module RSA_decrypt_128(
24
25     input wire clk,
26     input wire rst_n,
27     input wire start,
28
29     input wire [127:0] ciphertext,
30
31     output reg [127:0] plaintext,
32     output reg done
33
34 );

```

```

36 //=====
37 // Fixed RSA Private Key
38 //=====
39
40 parameter [127:0] N_CONST =
41     128'h9195a8ae3168963d09492e2d9420b23;
42
43 parameter [127:0] D_CONST =
44     128'hc29e9ef3891647fc8350c4132975aff9;
45
46 //=====
47 // FSM
48 //=====
49
50 parameter IDLE      = 2'd0;
51 parameter START_EXP = 2'd1;
52 parameter WAIT_EXP  = 2'd2;
53 parameter FINISH    = 2'd3;
54 reg [1:0] state;
55
56 //=====
57 // mod_exp interface
58 //=====
59
60 reg exp_start;
61 wire [127:0] exp_result;
62 wire exp_done;
63
64 //=====
65 // Modular Exponentiation
66 //=====
67
68 mod_exp_128 U_EXP (
69
70     .clk(clk),
71     .rst_n(rst_n),
72     .start(exp_start),
73
74     .a(ciphertext),
75     .b(D_CONST),
76     .n(N_CONST),
77
78     .result(exp_result),
79     .done(exp_done)
80 );
81
82 //=====
83 // FSM
84 //=====
85
86 always @(posedge clk or negedge rst_n) begin
87
88     if(!rst_n) begin
89
90         state <= IDLE;

```

```

92     plaintext <= 0;
93     done <= 0;
94
95     exp_start <= 0;
96 end
97 else begin
98
99     case(state)
100
101         //=====
102         // IDLE
103         //=====
104
105         IDLE: begin
106
107             done <= 0;
108             exp_start <= 0;
109
110             if(start) begin
111
112                 state <= START_EXP;
113             end
114         end
115
116         //=====
117         // START exponentiation
118         //=====
119
120         START_EXP: begin
121
122             exp_start <= 1'b1;
123
124             state <= WAIT_EXP;
125         end
126
127         //=====
128         // WAIT
129         //=====
130
131         WAIT_EXP: begin
132
133             exp_start <= 1'b0;
134
135             if(exp_done) begin
136
137                 plaintext <= exp_result;
138
139                 state <= FINISH;
140             end
141         end
142
143         //=====
144         // FINISH
145         //=====
146
147         FINISH: begin
148
149             done <= 1'b1;
150
151             state <= IDLE;
152         end
153
154         default: begin
155
156             state <= IDLE;
157         end
158     endcase
159 end
160 end
161 endmodule

```

Testbench :

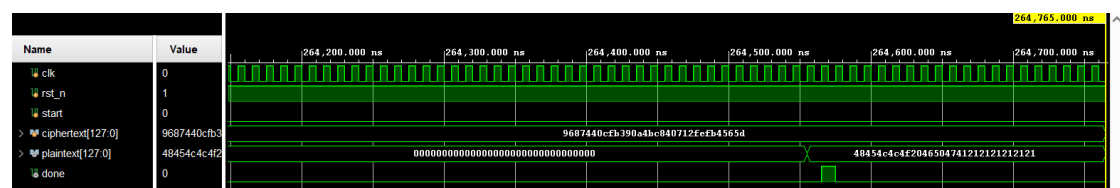
```

23 module tb_RSA_decrypt_128;
24     reg clk;
25     reg rst_n;
26     reg start;
27
28     reg [127:0] ciphertext;
29
30     wire [127:0] plaintext;
31     wire done;
32
33     RSA_decrypt_128 DUT (
34
35         .clk(clk),
36         .rst_n(rst_n),
37         .start(start),
38
39         .ciphertext(ciphertext),
40
41         .plaintext(plaintext),
42         .done(done)
43     );
44
45     initial begin
46         clk = 0;
47         forever #5 clk = ~clk;
48     end
49
50     initial begin
51
52         rst_n = 0;
53         start = 0;
54
55         ciphertext = 0;
56
57         #20;
58         rst_n = 1;
59
60         #20;
61
62         ciphertext =
63             128'h9687440cfb390a4bc840712fefb4565d;
64
65         start = 1;
66         #10;
67         start = 0;
68
69         wait(done);
70
71         #200;
72         $finish;
73     end
74 endmodule

```

Simulation :

Behavior Simulation :



7. Datapath

Verilog :

```
23 module datapath(  
24  
25     input wire clk,  
26     input wire rst_n,  
27  
28     input wire sipo_valid,  
29     input wire enc_start,  
30     input wire dec_start,  
31     input wire piso_start,  
32  
33     input wire [7:0] rx_data,  
34  
35     output wire sipo_done,  
36     output wire enc_done,  
37     output wire dec_done,  
38     output wire piso_done,  
39  
40     output wire [7:0] tx_data  
41 );  
42  
43 wire [127:0] plaintext_wire;  
44 wire [127:0] ciphertext_wire;  
45 wire [127:0] decrypted_wire;  
  
47 SIPO_16x8 U_SIPO(  
48     .clk(clk),  
49     .rst_n(rst_n),  
50     .in_valid(sipo_valid),  
51     .din(rx_data),  
52     .dout(plaintext_wire),  
53     .done(sipo_done)  
54 );  
55  
56 RSA_encrypt_128 U_ENC(  
57     .clk(clk),  
58     .rst_n(rst_n),  
59     .start(enc_start),  
60     .plaintext(plaintext_wire),  
61     .ciphertext(ciphertext_wire),  
62     .done(enc_done)  
63 );  
64  
65 RSA_decrypt_128 U_DEC(  
66     .clk(clk),  
67     .rst_n(rst_n),  
68     .start(dec_start),  
69     .ciphertext(ciphertext_wire),  
70     .plaintext(decrypted_wire),  
71     .done(dec_done)  
72 );  
73  
74 PISO_16x8 U_PISO(  
75     .clk(clk),  
76     .rst_n(rst_n),  
77     .start(piso_start),  
78     .din(decrypted_wire),  
79     .dout(tx_data),  
80     .done(piso_done)  
81 );  
82 endmodule
```

Testbench :

```

23 module tb_datapath();
24
25     reg clk;
26     reg rst_n;
27
28     reg sipo_valid;
29     reg enc_start;
30     reg dec_start;
31     reg piso_start;
32
33     reg [7:0] rx_data;
34
35     wire sipo_done;
36     wire enc_done;
37     wire dec_done;
38     wire piso_done;
39     wire [7:0] tx_data;
40
41     datapath DUT (
42         .clk(clk),
43         .rst_n(rst_n),
44         .sipo_valid(sipo_valid),
45         .enc_start(enc_start),
46         .dec_start(dec_start),
47         .piso_start(piso_start),
48         .rx_data(rx_data),
49         .sipo_done(sipo_done),
50         .enc_done(enc_done),
51         .dec_done(dec_done),
52         .piso_done(piso_done),
53         .tx_data(tx_data)
54     );
55
56     initial begin
57         clk = 0;
58         forever #5 clk = ~clk;
59     end
60
61     reg [127:0] plaintext;
62     integer i;
63
64     initial begin
65         rst_n = 0;
66         sipo_valid = 0;
67         enc_start = 0;
68         dec_start = 0;
69         piso_start = 0;
70         rx_data = 0;
71
72         plaintext = 128'h112233445566778899aabbccddeeff11;
73
74         #100;
75         rst_n = 1;
76         #20;
77
78         //=====
79         // 1. SIPO
80         //=====
81         for(i = 16; i > 0; i = i - 1) begin
82             @(negedge clk);
83             sipo_valid = 1'b1;
84             rx_data = plaintext[8*i-1 -: 8];
85         end
86         @(negedge clk);
87         sipo_valid = 1'b0;
88
89         // Wait SIPO done
90         wait(sipo_done);
91         @(negedge clk);
92

```

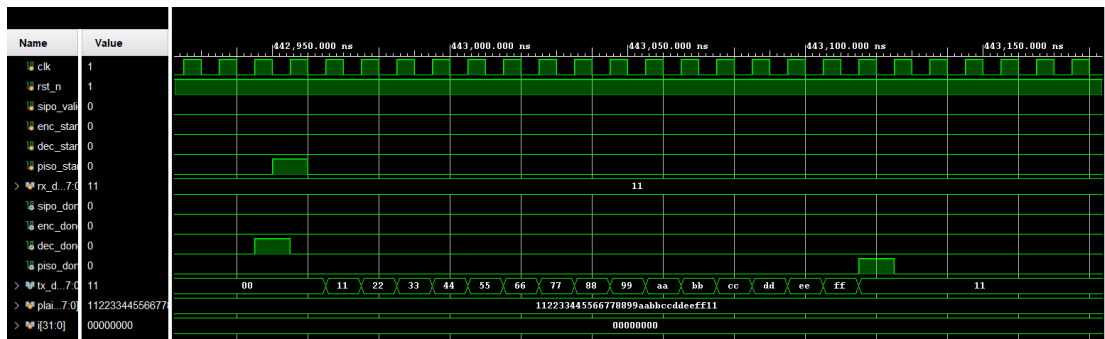
```

93 //=====
94 // 2. Start Encryption
95 //=====
96 enc_start = 1'b1;
97 @(negedge clk);
98 enc_start = 1'b0;
99
100 // Wait Encryption
101 wait(enc_done);
102 @(negedge clk);
103
104 //=====
105 // 3. Start Decryption
106 //=====
107 dec_start = 1'b1;
108 @(negedge clk);
109 dec_start = 1'b0;
110
111 // Wait Decryption
112 wait(dec_done);
113 @(negedge clk);
114
115 //=====
116 // 4. PISO
117 //=====
118 piso_start = 1'b1;
119 @(negedge clk);
120 piso_start = 1'b0;
121
122 wait(piso_done);
123 @(negedge clk);
124
125 #200;
126 $finish;
127 end
128 endmodule

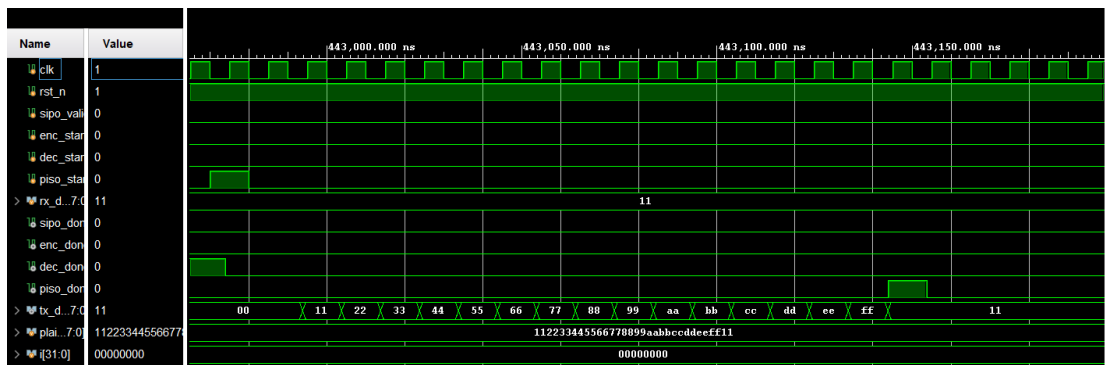
```

Simulation :

Behavior Simulation :



Post-Route Simulation :



8. Controller

Verilog :

```
23 module controller(  
24     input wire clk,  
25     input wire rst_n,  
26     input wire start,  
27     input wire sipo_done,  
28     input wire enc_done,  
29     input wire dec_done,  
30     input wire piso_done,  
31     output reg sipo_valid,  
32     output reg enc_start,  
33     output reg dec_start,  
34     output reg piso_start,  
35     output reg done  
36 );  
37  
38 parameter IDLE      = 4'd0;  
39 parameter LOAD_INPUT = 4'd1;  
40 parameter START_ENC  = 4'd2;  
41 parameter WAIT_ENC   = 4'd3;  
42 parameter START_DEC  = 4'd4;  
43 parameter WAIT_DEC   = 4'd5;  
44 parameter START_PISO = 4'd6;  
45 parameter WAIT_PISO  = 4'd7;  
46 parameter FINISH     = 4'd8;  
47  
48 reg [3:0] state;  
49  
50 always @(posedge clk or negedge rst_n) begin  
51  
52     if(!rst_n) begin  
53  
54         state <= IDLE;  
55         sipo_valid <= 0;  
56         enc_start <= 0;  
57         dec_start <= 0;  
58         piso_start <= 0;  
59         done <= 0;  
60     end  
61     else begin  
62  
63         // default pulse signals  
64         enc_start <= 0;  
65         dec_start <= 0;  
66         piso_start <= 0;  
67         done <= 0;  
68  
69         case(state)  
70
```

```

76 //=====
77 // IDLE
78 //=====
79
80 IDLE: begin
81
82     sipo_valid <= 0;
83
84     if(start) begin
85
86         state <= LOAD_INPUT;
87     end
88 end
89
90 //=====
91 // Receive 16 Bytes
92 //=====
93
94 LOAD_INPUT: begin
95
96     sipo_valid <= 1;
97
98     if(sipo_done) begin
99
100         sipo_valid <= 0;
101
102         state <= START_ENC;
103     end
104 end
105
106 //=====
107 // Start Encrypt
108 //=====
109
110 START_ENC: begin
111
112     enc_start <= 1'b1;
113
114     state <= WAIT_ENC;
115 end
116
117 //=====
118 // Wait Encrypt
119 //=====
120
121 WAIT_ENC: begin
122
123     if(enc_done) begin
124
125         state <= START_DEC;
126     end
127 end
128
129 //=====
130 // Start Decrypt
131 //=====
132
133 START_DEC: begin
134
135     dec_start <= 1'b1;
136
137     state <= WAIT_DEC;
138 end
139

```

```

140 //=====
141 // Wait Decrypt
142 //=====
143
144 WAIT_DEC: begin
145
146     if(dec_done) begin
147
148         state <= START_PISO;
149     end
150 end
151
152 //=====
153 // Start Output
154 //=====
155
156 START_PISO: begin
157
158     piso_start <= 1'b1;
159
160     state <= WAIT_PISO;
161 end
162
163 //=====
164 // Wait Output
165 //=====
166
167 WAIT_PISO: begin
168
169     if(piso_done) begin
170
171         state <= FINISH;
172     end
173 end
174
175 //=====
176 // FINISH
177 //=====
178
179 FINISH: begin
180
181     done <= 1'b1;
182
183     state <= IDLE;
184 end
185
186 default: begin
187
188     state <= IDLE;
189 end
190 endcase
191 end
192 end
193 endmodule

```

Testbench :

```

23 module tb_controller;
24     reg clk;
25     reg rst_n;
26
27     reg start;
28     reg sipo_done;
29     reg enc_done;
30     reg dec_done;
31     reg piso_done;
32
33     wire sipo_valid;
34     wire enc_start;
35     wire dec_start;
36     wire piso_start;
37     wire done;
38
39     controller DUT (
40         .clk(clk),
41         .rst_n(rst_n),
42         .start(start),
43         .sipo_done(sipo_done),
44         .enc_done(enc_done),
45         .dec_done(dec_done),
46         .piso_done(piso_done),
47         .sipo_valid(sipo_valid),
48         .enc_start(enc_start),
49         .dec_start(dec_start),
50         .piso_start(piso_start),
51         .done(done)
52     );
53
54     initial begin
55         clk = 0;
56         forever #5 clk = ~clk;
57     end
58
59     initial begin
60         rst_n = 0;
61         start = 0;
62         sipo_done = 0;
63         enc_done = 0;
64         dec_done = 0;
65         piso_done = 0;
66
67         #20;
68         rst_n = 1;
69         #20;
70
71         //=====
72         // Start system
73         //=====
74
75         @(negedge clk);
76         start = 1;
77         @(negedge clk);
78         start = 0;
79

```

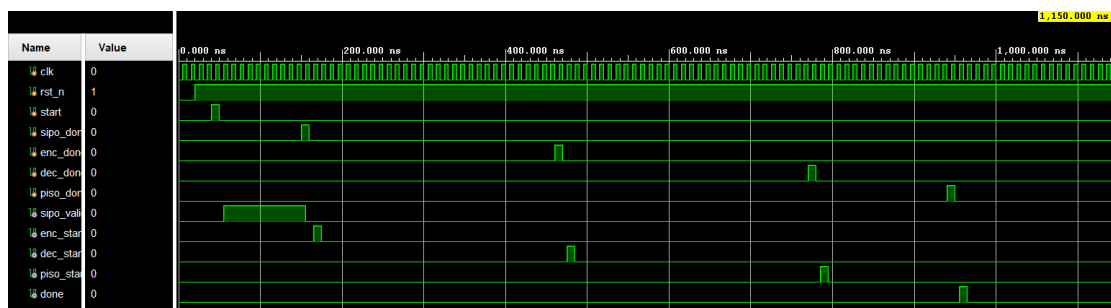
```

80 //=====
81 // SIPO done
82 //=====
83
84 // 模擬等待幾個 Clock 後 SIPO 完成
85 repeat(10) @(negedge clk);
86 sipo_done = 1;
87 @(negedge clk);
88 sipo_done = 0;
89
90 //=====
91 // Encryption done
92 //=====
93
94 // 模擬等待加密完成
95 repeat(30) @(negedge clk);
96 enc_done = 1;
97 @(negedge clk);
98 enc_done = 0;
99
100 //=====
101 // Decryption done
102 //=====
103
104 // 模擬等待解密完成
105 repeat(30) @(negedge clk);
106 dec_done = 1;
107 @(negedge clk);
108 dec_done = 0;
109
110 //=====
111 // PISO done
112 //=====
113
114 // 模擬等待 PISO 輸出完成
115 repeat(16) @(negedge clk);
116 piso_done = 1;
117 @(negedge clk);
118 piso_done = 0;
119
120 repeat(20) @(negedge clk);
121 $finish;
122 end
123 endmodule

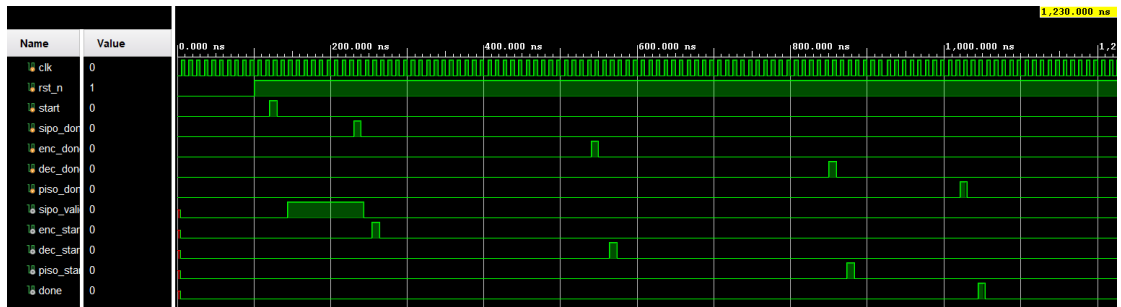
```

Simulation :

Behavior Simulation :



Post-Route Simulation :



9. Complete RSA System

Verilog :

```

23 module RSA_complete_128(
24
25     input wire clk,
26     input wire rst_n,
27     input wire start,
28     input wire [7:0] rx_data,
29     output wire [7:0] tx_data,
30     output wire done
31 );
32
33
34     wire sipo_valid;
35
36     wire enc_start;
37     wire dec_start;
38     wire piso_start;
39
40     wire sipo_done;
41     wire enc_done;
42     wire dec_done;
43     wire piso_done;
44
45     controller U_CONTROLLER(
46
47         .clk(clk),
48         .rst_n(rst_n),
49
50         .start(start),
51
52         .sipo_done(sipo_done),
53         .enc_done(enc_done),
54         .dec_done(dec_done),
55         .piso_done(piso_done),
56
57         .sipo_valid(sipo_valid),
58
59         .enc_start(enc_start),
60         .dec_start(dec_start),
61         .piso_start(piso_start),
62
63         .done(done)
64     );

```

```

66     datapath U_DATAPATH(
67
68         .clk(clk),
69         .rst_n(rst_n),
70
71         .sipo_valid(sipo_valid),
72
73         .enc_start(enc_start),
74         .dec_start(dec_start),
75         .piso_start(piso_start),
76
77         .rx_data(rx_data),
78
79         .sipo_done(sipo_done),
80         .enc_done(enc_done),
81         .dec_done(dec_done),
82         .piso_done(piso_done),
83
84         .tx_data(tx_data)
85     );
86 endmodule

```

Testbench :

```

23 module tb_RSA_complete_128;
24
25     reg clk;
26     reg rst_n;
27
28     reg start;
29     reg [7:0] rx_data;
30
31     wire [7:0] tx_data;
32     wire done;
33
34     RSA_complete_128 DUT(
35
36         .clk(clk),
37         .rst_n(rst_n),
38         .start(start),
39         .rx_data(rx_data),
40         .tx_data(tx_data),
41         .done(done)
42     );
43
44     initial begin
45         clk = 1'b0;
46         forever #10 clk = ~clk;
47     end
48
49     reg [127:0] plaintext;
50     integer i;
51
52     initial begin
53         rst_n = 1'b0;
54         start = 1'b0;
55         rx_data = 8'd0;
56
57         plaintext =
58             128'h123456789abcdeffedcba98765432101;
59
60         #100;
61         rst_n = 1'b1;
62         #40;

```

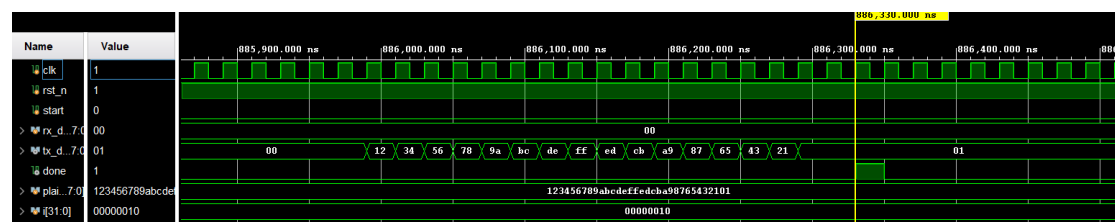
```

64 //=====
65 // Start System
66 //=====
67
68 @(posedge clk);
69 start <= 1'b1;
70 @(posedge clk);
71 start <= 1'b0;
72
73 //=====
74 // Wait Controller Enter LOAD_INPUT
75 //=====
76
77 #40;
78
79 //=====
80 // Send 16 Bytes
81 // MSB First
82 //=====
83
84 for(i = 0; i < 16; i = i + 1) begin
85
86     @(posedge clk);
87     rx_data <= plaintext[127 - i*8 -: 8];
88 end
89
90 //=====
91 // Stop Input
92 //=====
93
94 @(posedge clk);
95 rx_data <= 8'd0;
96
97 //=====
98 // Wait Finish
99 //=====
100
101 wait(done);
102
103 #300;
104 $finish;
105 end
106 endmodule

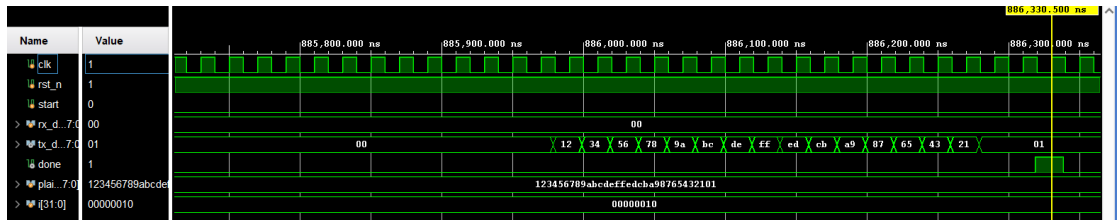
```

Simulation :

Behavior Simulation :



Post-Route Simulation :



Hardware cost

Clock Summary			
Name	Waveform	Period (ns)	Frequency (MHz)
clk	{0.000 5.800}	11.600	86.207

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.858 ns	Worst Hold Slack (WHS): 0.054 ns	Worst Pulse Width Slack (WPWS): 5.400 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 4788	Total Number of Endpoints: 4788	Total Number of Endpoints: 2420

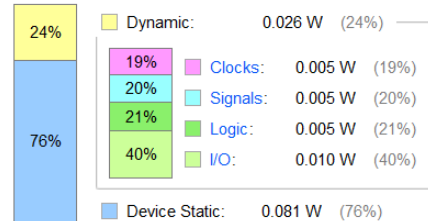
All user specified timing constraints are met.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	0.107 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	25.2°C
Thermal Margin:	59.8°C (31.6 W)
Ambient Temperature:	25.0 °C
Effective θ_{JA} :	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High
Launch Power Constraint Advisor to find and fix invalid switching activity	

On-Chip Power



Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												2672	2419
✓ impl_1	constrs_1	route_design Complete!	0.660	0.000	0.085	0.000		0.000	0.127	0	19 Warn			2668	2419

這次的 project 做的 128-bit RSA 加解密模組通過 vivado 佈線後驗證，在硬體資源花費上，整體優化良好，僅消耗 2668 個 LUT 與 2419 個 FF。系統可穩定運行於 86.2 MHz 的時脈頻率下，setup time (WNS: 3.858 ns)與 hold time (WHS: 0.054 ns)皆無任何 timing violation。

在功耗表現方面，透過匯入真實測資的佈線後模擬波形 SAIF 進行高可信度的功耗分析。分析結果顯示總功耗僅 0.107 W，其中動態功耗在 0.026 W。由功耗分佈可知，內部核心的邏輯與訊號翻轉佔比極低，動態功耗主要來自於必要的 I/O 資料傳輸。

Discussion

這次的 project 最具挑戰性的部分，我覺得是在於將高度複雜的 RSA 大數運算 (ex:模數乘法與指數運算) 轉化為具體的 RTL 電路。我並沒有採用軟體的寫法，因為老師有提醒如果使用 % 這個寫法來做取餘數的動作，這樣硬體合成出來會非常的大，所以我是採用 two adder 的方式一旦判斷到大於 n 時，就做 -n 的動作，來把範圍控制在 n 以內，再利用課本上學到的 modular exponential 演算法，把 128-bit 的模指數運算給設計出來。

在確認 datapath 之後，我設計了專屬的 controller (FSM) 來精準控制每一個運算步驟，同時，考量到外部介面的頻寬限制，我實作了 SIPO 與 PISO 模組，並透過嚴謹的握手訊號使各個模組之間可以完美連接。這讓我深刻體會到在複雜系統中，各模組間的握手與時序對齊的重要性。經過這次 128-bit RSA 的 project 磨練，讓我真正掌握了如何寫出功能正確且硬體友善的 verilog code。